FAKULTA MECHATRONIKY,
INFORMATIKY A MEZIOBOROVÝCH
STUDIÍ TUL

Diplomová práce

Architektura systému pohledové inspekce kvality

Studijní program: N0613A140028 – Informační technologie
Studijní obor: N0613A140028AI – Aplikovaná informatika
Autor práce: **Bc. Kevin Daněk**
Vedoucí práce: Ing. Igor Kopetschke

Liberec 2025

Tento list nahraďte originálem zadání.

PROHLÁŠENÍ

Prohlašuji, že svou diplomovou práci jsem vypracoval samostatně jako původní dílo s použitím uvedené literatury a na základě konzultací s vedoucím mé diplomové práce a konzultantem.
Jsem si vědom toho, že na mou diplomovou práci se plně vztahuje zákon č. 121/2000 Sb., o právu autorském, zejména § 60 – školní dílo.
Beru na vědomí, že Technická univerzita v Liberci nezasahuje do mých autorských práv užitím mé diplomové práce pro vnitřní potřebu Technické univerzity v Liberci.
Užiji-li diplomovou práci nebo poskytnu-li licenci k jejímu využití, jsem si vědom povinnosti informovat o této skutečnosti Technickou univerzitu v Liberci; v tomto případě má Technická univerzita v Liberci právo ode mne požadovat úhradu nákladů, které vynaložila na vytvoření díla, až do jejich skutečné výše.
Současně čestně prohlašuji, že text elektronické podoby práce vložený do IS STAG se shoduje s textem tištěné podoby práce.
Beru na vědomí, že má diplomová práce bude zveřejněna Technickou univerzitou v Liberci v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů.
Jsem si vědom následků, které podle zákona o vysokých školách mohou vyplývat z porušení tohoto prohlášení.

24. 3. 2026 Bc. Kevin Daněk

ARCHITEKTURA SYSTÉMU POHLEDOVÉ INSPEKCE KVALITY

ABSTRAKT

Tato zpráva popisuje třídu `tulthesis` pro sazbu absolventských prací Technické univerzity v Liberci pomocí typografického systému LaTeX.

Klíčová slova: LaTeX, třída, TUL

ABSTRACT

This report describes the `tulthesis` package for Technical university of Liberec thesis typesetting using the LaTeX typographic system.

Keywords: LaTeX, class, TUL

PODĚKOVÁNÍ

Dál bych chtěl poděkovat i zbytku akademické obce, se kterou jsem měl co dočinění. I když se s některými jejími členy, ozvláště pak z nadřízených pozic, neshodnu ani na počasí, tak díky každému jsem získal novou zkušenost. Někdy byla pozitivní, a někdy zase i negativní - to ale k životu patří.
A na závěr musím poděkovat svojí drahé polovičce, Simče Daňkové, která mě podporovala po celou dobu psaní této práce. Na začátku této práce byla mou přítelkyní, a nyní, na jejím konci, je již mojí manželkou. Díky ní si budu tuto práci pamatovat nejenom jako svůj *magnum opus*, ale jako životní milník, kterým se změnil skoro každý aspekt mého života. Pokud Vás zajímají regionální dějiny a příběhy z časů minulých, doporučuji si přečíst její bakalářskou práci, kterou tvořila souběžně s tou mojí, a zabývá se tématem sebevražd po druhé světové válce na Liberecku a obzvláště v Jablonci nad Nisou a jeho blízkém okolí.

OBSAH

Obsah 6
Seznam zkratek 8

1 Úvod **9**

2 Softwarové inženýrství **10**
2.1 Technický dluh 11
2.1.1 Vznik technického dluhu 12
2.1.2 Nešváry 12
2.1.3 Detekce technického dluhu 14
2.1.4 Řešení technického dluhu 14
2.2 Clean Code 16
2.2.1 Identifikátory 16
2.2.2 Funkce 17
2.3 Návrhové vzory 19
2.3.1 Factory 19
2.3.2 Builder 19
2.3.3 Singleton 19
2.3.4 Adapter 19
2.3.5 Facade 19
2.3.6 Proxy 19
2.3.7 Composite 19
2.3.8 Chain of Responsibility 19
2.3.9 Strategy 19
2.3.10 Observer 19
2.4 Architektonické vzory 19
2.5 Integrační vzory 19

3 Udržitelnost produkčního kódu **20**
3.1 Testování 20
3.2 Verzování 20
3.3 Postupná integrace 20
3.4 Postupné nasazení 20
3.5 Monitoring 20
3.5.1 Logy 21
3.5.2 Metriky 21

4 Audit architektury systému **22**
4.1 Modelování softwarové architektury 23
4.1.1 Diagram systémového kontextu 23
4.1.2 Diagram kontejnerů 24
4.1.3 Diagram komponent 24
4.2 Modelování doménových procesů 24
4.3 Modelování procesu nasazení 24
4.4 Vyhodnocení rizik 24

5 Architektura nového řešení **27**

6 Závěr **28**

SEZNAM ZKRATEK

CD	Continuous Deployment (Postupné nasazení)
CI	Continuous Integration (Postupná integrace)
DevOps	Development Operations
GPG	GNU Privacy Guard
JSON	Javascript Object Notation
JWT	JSON Web Tokens
SecOps	Security Operations
SSH	Secure Shell
TPM	Trusted Platform Module

1 ÚVOD

Každý, jakkoliv velký, systém musí někdo navrhnout. S rostoucí složitostí systému zároveň roste i složitost jeho správného návrhu. Ve své předchozí práci jsem se věnoval rapidnímu vývoji algoritmů strojového vidění, jehož výsledkem byla nová funkční komponenta pro trénování a inferenci neuronových sítí a jiných algoritmů strojového vidění. (Daněk, 2024)

Již tehdy byla samotná implementace omezena návrhem existujícího systému. Upřednostňování rychlého dodání nových funkcionalit na úkor průběžné údržby kódu vyústilo v enormní množství technologického dluhu, který se za sedm let ve zdrojovém kódu nastřádal. Tato práce navazuje na mou předchozí práci a dívá se na ní ze širší perspektivy. Cílem je odklonit se od vývoje nových funkcí a obrátit pozornost právě k zanedbané architektuře celého řešení. Rozsah analýzy jsem stanovil nad rámec softwarového návrhu a věnuji se rovněž architektuře infrastruktury nasazeného řešení.

2 SOFTWAROVÉ INŽENÝRSTVÍ

Programátor vybavený klávesnicí má téměř neomezené možnosti realizace svých nápadů, podobně jako malíř se štětcem. Je však pozoruhodné, že na rozdíl od nekontrolovaného tvůrčího procesu malíři často své možnosti úmyslně limitují. Právě dodržením určitých kompozičních pravidel totiž docílí toho, že obraz není pouze chaotickou směsí čar a barev, ale srozumitelnou myšlenkou, kterou divák dokáže instinktivně rozpoznat.

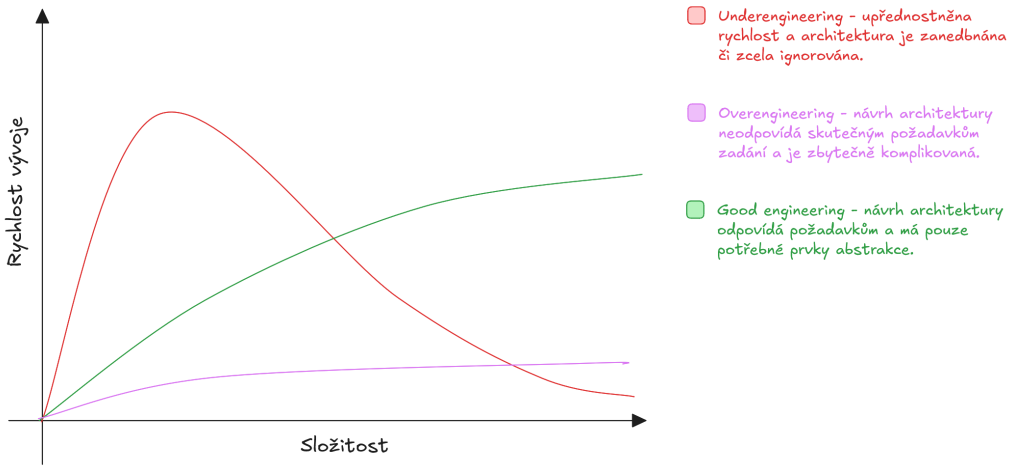
Principy vývoje softwaru se od této úvahy příliš neliší. Pokud se při vývoji zavážeme k dodržování stanovených pravidel a konvencí, umožníme ostatním vývojářům snadněji pochopit záměr našeho kódu. Této oblasti se právě věnuje **softwarové inženýrství**, které se zabývá návrhem kódu, který je udržitelný, čitelný, spravovatelný a testovatelný (Michigan Technological University, 2025). Z předvídatelného a čitelného kódu profituje i samotný autor, který se tak při návratu k vlastní práci po delším časovém odstupu nemusí věnovat náročnému dekódování původního záměru.

Softwarové inženýrství je velmi rozsáhlý obor a není v možnostech této práce jej pojmout v jeho úplnosti. Teoretický rámec se proto omezuje na vybraná témata, která jsou klíčová pro pochopení praktické části. Kapitolu otevírá pojednání o technickém dluhu, jeho příčinách a metodách jeho eliminace. Na tento koncept navazuje pasáž věnovaná principům udržitelného a testovatelného kódu, které tvoří nezbytný základ pro následné návrhové vzory. Ty využívají zmíněné principy a transformují je do osvědčených řešení běžných architektonických problémů.

Další část práce se věnuje metodice efektivního testování a principům kontinuální integrace a doručování (CI/CD). Teoretický blok uzavírá kapitola o monitoringu nasazeného řešení, který poskytuje nezbytný vhled do chování systému v produkčním prostředí, kde vývojář zpravidla nemá k dispozici stejné diagnostické nástroje jako v lokálním vývojovém prostředí.

2.1 TECHNICKÝ DLUH

Při vývoji software musí být učiněna určitá rozhodnutí o jeho návrhu. Ta jsou ovlivněna mnoha faktory - od hloubky znalosti domény a specifik zadání až po technologická omezení a časový tlak. Výsledná architektura je pak nevyhnutelně kompromisem mezi okamžitou rychlostí dodání a dlouhodobou udržitelností systému. Pod tlakem termínů musí vývojář neustále vyvažovat míru investice do čitelnosti, abstrakce a budoucí rozšířitelnosti kódu na úkor času, který má aktuálně k dispozici.



Obrázek 2.1: Rychlost vývoje v závislosti na složitosti návrhu

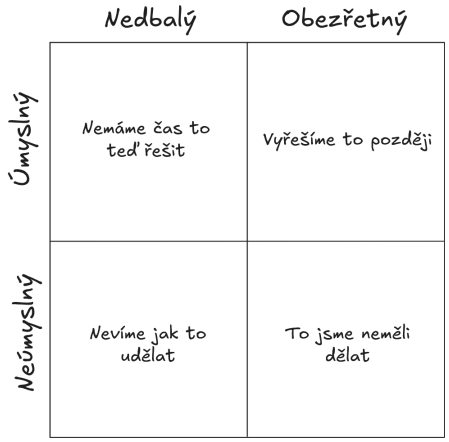
Pokud je zvolena cesta rychlejšího dodání namísto systematického návrhu, vzniká kód, který je tvořen ad hoc a postrádá ucelenou vizi. Absence celkového nadhledu pak způsobuje, že se systém s každou další funkcí stává nepřehlednějším a jeho údržba vyžaduje stále větší úsilí. Tento jev nazývám technickým dluhem. Pokud se technický dluh nechá střádat, postupně transformuje triviální změny na netriviální.

Trivialní změnou myslím takovou změnu, kdy není potřeba zasahovat do řídících struktur a logiky existujícího systému. Naopak netriviální změny vyžadují hlubší pozorumnění a studium existujícího kódu a jsou tím pádem nejen časově náročnější, ale i více zatíženy možnostmi pro vznik chyby. Příkladem triviální změny je přidání záznamu do slovníku dostupných algoritmů, nebo přidání nové obslužné metody pro údalost. Naopak příklad netriviální změny je aktualizace výrazů *switch* nebo *if* pro přidání nového algoritmu. V tento okamžik je totiž potřeba hlídat, že jsou tyto řídící struktury aktualizovány na všech relevantních místech.

Technický dluh se nazývá dluhem právě proto, že vykazuje stejné charakteristiky jako finanční závazek. Výměnou za rychlejší uvedení produktu na trh si vývojový tým od systému obrazně vypůjčuje čas, a to na úkor technické preciznosti. Stejně jako u finančních závazků, i zde je nutné počítat s úroky v podobě degradace kvality kódu.

2.1.1 Vznik technického dluhu

Vznik technického dluhu popisuje ve svých knihách a článcích Martin Fowler, který rozděluje příčiny vzniku technického dluhu do čtyř kategorií. Tyto kategorie leží na kvadrantu, který je založen na dvou osách: ose obezřetnosti a ose úmyslnosti. (Fowler, 2009)



Obrázek 2.2: Kvadranty technického dluhu (Team Asana, 2025)

Úmyslný dluh vzniká vědomým rozhodnutím upřednostnit rychlost dodání před správným návrhem. Naproti tomu neúmyslný dluh bývá důsledkem nezkušenosti nebo nedostatku znalostí v době tvorby architektury. Osa obezřetnosti pak odráží schopnost týmu zhodnotit rizika při zavádění technického dluhu: zatímco obezřetný dluh je přijat s plným vědomím budoucích nákladů na jeho splacení, nedbalý dluh vzniká s nedostatkem informací či zkušeností (Fowler, 2009).

2.1.2 Nešváry

Pokud je technický dluh podle mé dřívější definice jev způsobující celkovou nepřehlednost a neudržitelnost systému, vyvstává rozumná otázka - jak konkrétně technický dluh vypadá? Technický dluh je tvořen hromaděním kusů kódu, které degradují kvalitu softwarového řešení. Tyto kusy se nazývají *code smells*, a v této práci je nazývám **nešváry**.

Před samotným výčtem důležitých nešvárů je vhodné se pozastavit nad jejich tříděním a podívat se, jakým způsobem je odborníci klasifikují. Pro účely této práce je důležité zmínit dvě klasifikace, které na problém nahlíží zcela odlišně a přivádí do problematiky důležité myšlenky - Mäntylovu klasifikovaci a Perezovu klasifikaci. Mäntylä nahlíží na nešváry jako na skupiny příbuzných symptomů. Dívá se tedy na nešváry optikou jejich dopadu na kód samotný. Jednotlivé kategorie definoval na základě společného negativního dopadu na kvalitu kódu (Mäntylä, 2010; Alexander Shvets, 2018a):

- **Nabobtnalci** - tedy kód, který se rozrostl do takových proporcí, že s ním nelze nadále efektivně pracovat;
- **Zneužívači** - tedy kód, který špatně využívá možnosti objektového návrhu;
- **Zabraňovači** - tedy kód, který ztěžuje změny či další vývoj;
- **Zbytečníci** - tedy zbytečné části kódu, které ztěžují jeho pochopení;
- a **Vazbiči** - tedy případy, kdy je kód nesprávně provázaný a dochází ke špatnému rozdělení odpovědnosti.

Alternativní klasifikaci nabízí právě Perez, který netřídí nešváry do skupin podle společných vlastností, ale přišel s myšlenkou, že tyto nešváry tvoří de-facto monoid s operací skládání. Perez totiž popisuje některé nešváry jako nízkoúrovňové a atomické, tudíž jsou ireducibilní a tvoří základní kámeny pro skládání složitějších nešvárů, které nazývá vysokoúrovňovými (Perez, 2011). Tento pohled na nešváry závádí do problematiky důležitou myšlenku - že nešváry nejsou disjunktními problémy, ale že i v případě Mäntylovy klasifikace existuje míra překrytí. Pokud například odhalíme duplicitní kód, může být v tom samém místě přítomný i *nešvár divergentní změny* nebo *shotgun surgery*.

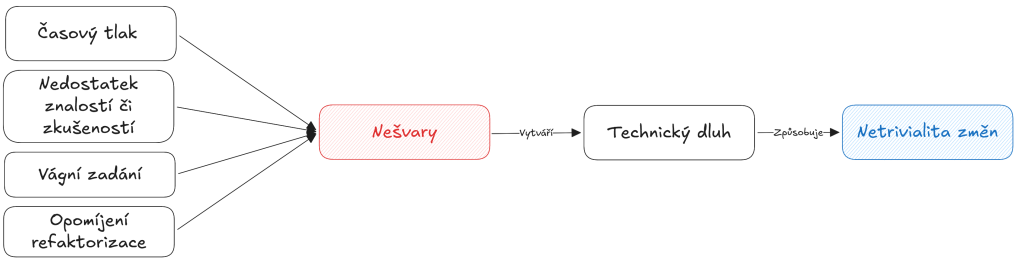
Samotný výčet nešvárů je dlouhý a každý zdroj bude jmenovat jinou podmnožinu. Níže jsem do tabulky sepsal podle mého názory ty nejpodstatnější nešvary, na které lze narazit, z původního seznamu podle Martina Fowlera (Alexander Shvets, 2018a).

Tabulka 2.1: Výběr nešvarů, jejich symptonů a příčin

Název	**Symptomy**	**Příčiny**
Dlouhá metoda	Metody nad 20 řádků.	Postupné nabalování logiky.
Velká třída	Třída s mnoha zodpovědnostmi.	Nedodržení principu jedné odpovědnosti.
Divergentní změna	Mnoho úprav v jednom souboru.	Kopírování kódu.
Shotgun Surgery	Jedna úprava v mnoha souborech.	Jedna odpovědnost v mnoha třídách.
Duplicitní kód	Stejná logika na více místech.	Kopírování kód.
Mrtvý kód	Nevyužité metody/proměnné.	Strach ze smazání kódu.
Switch Statements	Rozsáhlé switch/if řetězce.	Neobratné využití OOP.
Refused Bequest	Nevyužívání děděných metod.	Chybná hierarchie tříd.

2.1.3 Detekce technického dluhu

Každý nešvár přispívá k celkovému technickému dluhu. Nešváry jsou příčinou, technický dluh stavem a netrivialita změn následkem. V důsledku se tak technický dluh v projektu projevuje jako překážka v životním cyklu softwaru, kterou je nutné nejprve identifikovat, zmapovat a zpětně dotrasovat k nešvárům.



Obrázek 2.3: Technický dluh je kolekce nešvárů

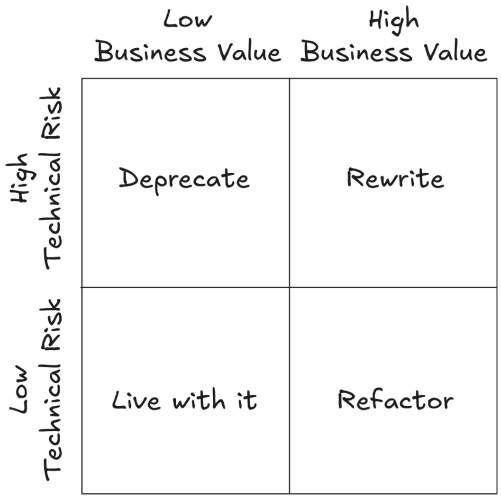
Prvotním indikátorem přítomnosti technického dluhu je obvykle dynamika samotného vývoje. Pokud se rychlost vývoje snižuje a čas potřebný k implementaci nových funkcionalit či opravě zdánlivě prostých chyb roste, bývá jednou z příčin právě nutnost vývojářů neustále překonávat bariéry v podobě zanedbaného či nepřehledného kódu. Právě samotní vývojáři nabízí sice subjektivní, avšak kriticky důležité hodnocení. Ti jsou totiž schopni přesně identifikovat konkrétní úseky systému, jež je při práci nejvíce zpomalují.

Objektivnější pohled nabízejí nástroje pro statickou analýzu kódu, které dokáží automatizovaně detekovat duplicity nebo antipatterny. Významnou metriku představuje rovněž code churn – ukazatel četnosti změn v konkrétních částech kódu. Vysoká míra netriviálních změn v určitém modulu často signalizuje jeho nevhodný návrh, nestabilitu nebo přílišnou provázanost s ostatními částmi systému, což jej činí náchylným k chybám a náročným na údržbu (Team Asana, 2025).

2.1.4 Řešení technického dluhu

Volba strategie pro řešení technického dluhu vyžaduje analýzu rozsahu, souvisejících rizik a dostupných zdrojů. U systémů s nízkou prioritou, kde případná chybovost nepředstavuje zásadní riziko, nemusí být eliminace dluhu ekonomicky efektivní a lze jej v systému ponechat. Naopak u kritických komponent, které vykazují vysokou poruchovost a nízkou udržitelnost, je nutné zvolit aktivní přístup. Rozhodovací proces se v takovém případě soustředí na volbu mezi postupnou refaktorizací, která zachovává funkční celek a snižuje riziko regrese, a kompletním přepisem. Zatímco refaktorizace je vhodná pro kontinuální splácení dluhu, úplný přepis je často nevyhnutelný tehdy, pokud je stávající architektura již natolik rigidní nebo technologicky

zastaralá, že jakákoliv snaha o její údržbu a rozšiřování postrádá ekonomický i časový smysl.



Obrázek 2.4: Kvadranty řešení technického dluhu (ForrestKnight, 2025)

Následující kapitoly se budou věnovat technikám a návrhovým principům, které mají za úkol snížit množství technického dluhu na jednotlivých úrovních abstrakce či mu úplně předejít.

2.2 CLEAN CODE

Clean Code představuje soubor pravidel a doporučení, jejichž primárním cílem je snížit kognitivní zátěž při čtení a údržbě zdrojového kódu (Martin; Coplien, 2009). Na clean code je pohlíženo jako na myšlenkový pochod, který má za cíl sjednotit komunikaci záměrů a zjednodušit údržbu či rozšíření kódu po určitém časovém odstupu. Jak píše i samotný Robert Martin ve své knize[1], jednotlivé zásady nejsou absolutní pravdou a míra jejich aplikace může být předmětem diskuse v týmu. V následujících podkapitolách jsou rozebrány vybrané zásady, které mají podle mých zkušeností největší vliv na čitelnost a dlouhodobou udržitelnost systému.

Je důležité zdůraznit, že cílem Clean Code není přesné dodržování arbitrárních pravidel. Smyslem je zmírnit negativní dopady neudržitelného kódu – často psaného pod časovým tlakem – na budoucí produktivitu celého týmu. Žádoucím výsledkem je udržení konstantního tempa vývoje v čase a eliminace potřeby rozsáhlých a rizikových přepisů, či celkové refaktorizace projektu.

2.2.1 Identifikátory

Vývoj softwaru vyžaduje pojmenovávání široké škály konstrukcí – od proměnných a funkcí až po třídy, rozhraní a specifické jazykové struktury. Zásadním principem je, aby každý název nesl jasný význam a záměr. Název identifikátoru by měl zodpovídat otázky jako: „Proč tato entita existuje?“, „Co dělá?“ a „Jakým způsobem se používá?“. Například jednopísmenný název proměnné neposkytuje žádnou informaci o jejím účelu. Pokud však řešíme výpočet délky přepony pravoúhlého trojúhelníku, název `delkaPreponyCm` jasně definuje obsah i jednotky dané hodnoty (Martin; Coplien, 2009).

Ke srozumitelnosti názvu patří i zdánlivě jasné pravidlo. Název nesmí lhát. Nenazývejme věc seznamem, pokud to skutečně není seznam. Zatímco od seznamu programátor očekává možnost indexace přirozenými čísly, u slovníku je tento předpoklad chybný a kód se stává neintuitivním. S tím souvisí i nebezpečí příliš podobných názvů. Ve vývojových prostředích, která nabízejí našeptávání kódu nebo automatické doplňování pomocí AI asistentů, může minimální rozdíl v názvech vést k fatálním, těžko odhalitelným, chybám (Martin; Coplien, 2009).

Jednotlivé identifikátory by se měly lišit natolik, aby bylo možné sledovat roli, kterou v systému plní. Pojmenování argumentů funkce stejným slovem s numerickým sufixem (např. `data1`, `data2`) neposkytuje žádnou informaci o jejich sémantickém významu. Kromě číslovek se lze často setkat s přístupem přidání afixů jako `manager` nebo `data`. Pokud názvy od sebe odlišíme akorát těmito afixy, ztrácí se smysl daných proměnných.

[1] MARTIN, Robert C.; COPLIEN, James O., 2009. *Clean code: a handbook of agile software craftsmanship*. Prentice Hall. ISBN 9780132350884.

Mnohdy totiž vývojář použije název s afixem jenom proto, že identifikátor již v kontextu existuje a nenapadá ho lepší název. Pokud dva identifikátory nesou různý význam, musí být i jejich názvy různé (Martin; Coplien, 2009).

Identifikátory by měli být hledatelné a vyslovitelné. Pokud má vývojový tým o kódu efektivně diskutovat, musí být názvy použitelné v běžné řeči bez nutnosti hláskovat nesrozumitelné zkratky. Vyhledatelnost pak úzce souvisí s unikátností názvu v rámci kontextu. Vyhneme se tím dlouhému přemýšlení nad tím, co daná zkratka znamená, a můžeme se místo toho soustředit na podstatu věci. V této souvislosti je rovněž žádoucí vyhýbat se tzv. „magickým konstantám“ v řídicích strukturách a místo nich definovat pojmenované konstanty na odpovídající úrovni abstrakce (Martin; Coplien, 2009).

Při snaze o odlišení je však nutné vyvarovat se kódování technických informací (např. datového typu) přímo do názvu, což je relikt dřívějších dob (tzv. maďarská notace). V éře statické analýzy není nutné, aby název proměnné obsahoval informaci o tom, že jde o `int`, nebo aby název rozhraní začínal písmenem `I`. Kolize názvu rozhraní a třídy (např. `Camera`) by neměla být řešena prefixem `ICamera`, ale raději sémantickým upřesněním konkrétní implementace – například `CameraService` nebo `PhysicalCamera`. Takový přístup do názvu vkládá informaci o roli či principu fungování, nikoliv pouze o jazykové konstrukci (Martin; Coplien, 2009).

2.2.2 Funkce

Funkce a procedury jsou většinou první úrovní abstrakce a metodou pro členění kódu. *Clean Code* proto shrnuje pravidla pro návrh funkcí do tří principů: funkce by měla být krátká, měla by vykonávat právě jednu činnost a měla by svoje působení omezit na kontext funkce (nesmí mít *vedlejší účinky*) (Martin; Coplien, 2009).

Požadavek na minimalizaci délky funkce je především varováním před přílišnou hloubkou zanoření. Vysoká míra zanoření v řídicích strukturách obvykle indikuje nekonzistentní úroveň abstrakce a naznačuje, že by konkrétní bloky kódu měly být refaktorovány do samostatných metod. Jak poznamenal Linus Torvalds: *„if you need more than 3 levels of indentation, you're screwed anyway, and should fix your program.“* (Torvalds, 2007)

Princip jediné činnosti na úrovni funkce znamená, že by v rámci jejího těla měla být zachována jednotná úroveň abstrakce. Funkce by měla buď vykonávat příkaz, nebo odpovídat na dotaz, ale nikdy obojí zároveň (princip *Command Query Separation*). Vysokoúrovňová logika by neměla být zatěžována nízkoúrovňovými výpočty; ty by měly být delegovány na specializované podfunkce. Tento hierarchický přístup lze přirovnat k fungování firmy: ředitel se věnuje strategickému řízení a logistické procesy deleguje na vedoucí skladů, kteří dále řídí konkrétní naskladnění zboží (Martin; Coplien, 2009).

Pro udržení konzistentní abstrakce je doporučeno dodržovat pravidlo sestupného čtení (The Stepdown Rule). Program by měl být čitelný jako posloupnost odstavců, kde každý rozvíjí detaily předchozího, uvozený pomyslnou předponou „k“ (v anglickém originále *to*):

- K vyřízení objednávky je třeba zabalit zboží, odečíst jej ze skladu a vytisknout štítek.
- K zabalení zboží je třeba položku vyhledat a zvolit vhodný obal.
- K odečtení ze skladu je třeba identifikovat pozici zboží v registru.
- K vytisknutí štítek je třeba určit přepravce a vygenerovat tisková data.

Komplexitu funkce však neurčuje pouze její tělo, ale také počet jejích argumentů. Ideální funkce je niladická (bez argumentů). Běžné jsou funkce monadické (jeden argument) a dyadické (dva argumenty). Přítomnost triadické (tři argumenty) nebo dokonce polyadické funkce (čtyři a více) je vnímána jako varovný signál nevhodného návrhu (Martin; Coplien, 2009).

S rostoucím počtem parametrů roste také počet testovacích případů, pokud předpokládáme čtyři parametry, které jsou pravdivostní hodnoty, jedná se o $2^4 = 16$ kombinací, které je potřeba otestovat. Podobně můžeme postupovat i u spojitých proměnných, kde vybíráme zástupce z jednotlivých tříd ekvivalence. Menší počet argumentů nejenom zjednodušejí použití funkce, ale i její testování.

S rostoucím počtem parametrů roste i složitost testování. Uvažujeme-li funkci se čtyřmi parametry typu *boolean*, vzniká celkem $2^4 = 16$ kombinací vstupních stavů, které je nutné validovat. U spojitých proměnných je situace ještě složitější a vyžaduje pečlivý výběr zástupců z tříd ekvivalence. Rostoucí počet argumentů také zavádí problém, kdy mezi nimi chybí přirozené pořadí. Hrozí tak riziko záměny, pokud nás vlastní intuice nenavede správně[2].

Pokud funkce vyžaduje velký počet parametrů, je vhodné zvážit jejich seskupení do logických datových struktur nebo objektů. *Clean Code* rovněž důrazně varuje před používáním tzv. příznakových argumentů (flag arguments). Pokud funkce přijímá *boolean*, který větví tok funkce, z definice vykonává více než jednu věc a měla by být rozdělena do dvou samostatných funkcí.

[2]Existují situace, kdy mezi parametry je přirozené uspořádání, například souřadnice v kartézské soustavě, kdy jsou souřadnice tvořeny uspořádanou dvojicí (x, y)

2.3 NÁVRHOVÉ VZORY

Návrhové vzory představují konceptuální řešení opakujících se problémů v rámci návrhu softwarové architektury. Na rozdíl od algoritmu, který definuje jasně předepsanou sekvenci kroků k dosažení konkrétního výsledku, jsou návrhové vzory spíše obecnými šablonami či strategiemi. Jednotlivé vzory vývojář implementuje v souladu se specifiky daného programovacího jazyka a potřebami konkrétního programu (Alexander Shvets, 2018b).

Zatímco principy Clean Code se soustředí na technické řemeslo a disciplínu na té nejnižší úrovni – tedy na to, jak psát čitelný a udržitelný kód – návrhové vzory představují strategie, které určují, co a proč stavět. Tyto vzory přenášejí žádoucí vlastnosti Clean Code do vyšších úrovní abstrakce, kde se formují rozsáhlejší architektonické celky. Cílem této kapitoly není taxativní výčet všech existujících vzorů, nýbrž analýza konkrétních problémů, které tyto vzory adresují, a mechanismů, jimiž dosahují kýženého řešení. Zvláště je dobré umět identifikovat vhodnost konkrétního vzoru ještě před napsáním prvního řádku kódu, čímž lze, alespoň z části, předejít vzniku dalšího technického dluhu.

2.3.1 Factory

2.3.2 Builder

2.3.3 Singleton

2.3.4 Adapter

2.3.5 Facade

2.3.6 Proxy

2.3.7 Composite

2.3.8 Chain of Responsibility

2.3.9 Strategy

2.3.10 Observer

2.4 ARCHITEKTONICKÉ VZORY

2.5 INTEGRAČNÍ VZORY

3 UDRŽITELNOST PRODUKČNÍHO KÓDU

3.1 TESTOVÁNÍ

3.2 VERZOVÁNÍ

3.3 POSTUPNÁ INTEGRACE

3.4 POSTUPNÉ NASAZENÍ

3.5 MONITORING

Kritickou součástí vývoje softwaru je jeho testování a ladění (debugging). Testy, například jednotkové nebo integrační, simulují chování systému za různých podmínek a následné ladění pomocí debuggerů odhaluje jeho vnitřní stav instrukce po instrukci. V praxi ovšem není možné odhalit všechny chyby a dojde na situace, kdy se v nasazeném řešení chyba objeví. Při lokálním vývoji by se program odkrokoval debuggerem a zjistili se okolnosti, které chybu způsobily.

V produkčním nasazení ovšem luxus odkrokování programu zpravidla nemáme, ať už kvůli chybějícím pomocným strukturám (například *symbol table*) a nebo kvůli optimalizacím kompilátoru, který program převede do jazyka symbolických adres či byte kódu. Když tedy nelze pozorovat stav programu zevnitř, nezbývá nám nic jiného, než ho pozorovat zvenčí - a konkrétně přes jeho výstupy. Schopnost pozorovat chování systému na základě jeho výstupů nazýváme **observabilitou**.

Mnoho autorů a spolků, kteří se observabilitou zabývají, jí rozdělují na pilíře. Cílem těchto pilířů je zodpovědět na otázky: *Co se to děje?*, *Proč se to děje?* a *Kde se to děje?*. Každý pilíř tak doplňuje informace k těm dalším, a dohromady tvoří kompletní vhled do chování systému. (Elastic Observability Team, 2024) Tato práce ovšem bude na observabilitu nahlížet trochu jinak. Paradigma pilířů indikuje, že každý pilíř je disjunktním článkem, které, když se spojí dohromady, tvoří observabilitu.

3.5.1 Logy

Jedním z nejčastějších výstupů programu, který se používá pro ladění nasazeného řešení, jsou logy. Jedná se o textové záznamy, které nesou informaci o událostech v programu. Logování má své zásady a doporučení, jejichž dodržení či nedodržení ovlivňuje míru, kterou nám logy pomohou s diagnostikou problému. Ačkoliv je logování velmi užitečné, nedává nám komplexní obraz o chování systému, protože se jedná jenom o jeden z možných výstupů programu. Program může vytvářet další telemetrická data, která mohou být následně dále zpracována.

3.5.2 Metriky

Metriky jsou kvantitativní veličiny, které měří konkrétní vlastnosti běžícího programu. Můžeme tak měřit například počet požadavků na konkrétní instanci či počet I/O operací. Metriky máme přímo měřené a odvozené, přičemž odvozené metriky můžeme zjišťovat z logů (například počet chyb v čase)

Je nutné podotknout, že metriky má smysl zaznamenávat dlouhodobě jakožto časové řady, abychom určili chování systému v čase. Je nutné tak metriky zaznamenávat na základě události, nebo pravidelného měření. Naopak zaznamenávat metriky vycházející ze statické analýzy smysl nedává, protože pokud bychom chtěli například analyzovat složitost programu pomocí Halsteadových vzorců, tak se tato metrika nemůže měnit v čase běhu programu. Naopak zatížení konkrétní instance programu nebo využití hardwarových prostředků proměnlivé čase běhu programu je.

4 AUDIT ARCHITEKTURY SYSTÉMU

Tato kapitola se zabývá analýzou existujícího řešení. Cílem je stávající řešení zmapovat, modelovat pomocí diagramů, a popisovat ho na dostatečně detailních úrovní abstrakce, která mi dovolí poukázat na přítomný technický dluh. K zmapování a rozdělení na úrovně abstrakce jsem využil C4 diagramy, které navrhnul Simon Brown k popisu architektury na celkem čtyřech úrovních abstrakce (Brown, 2024a).

Ve světe softwarového inženýrství je již zavadený standard, který slouží k modelování softwarové architektury, a to je *Unified Modelling Language*. Ten slouží k modelování a dokumentaci jednotlivých částí a procesů. Modeluje tak jak vztahy mezi třídami, tak zároveň umožňuje zaznamenat komunikaci mezi komponentami pomocí sekvenčního diagramu či cestu uživatele pomocí user journey diagramu (Špánek, 2022). Proč jsem tedy nezvolil tento standard?

Jak popisuje Simon Brown v úvodu do C4 diagramů, týmy očekávají jednoduchost a srozumitelnost zápisu (Brown, 2024a). UML diagramy slouží zároveň jako dokumentace, což je jeho síla, a zároveň jeho slabina. Dokumentace musí být dostatečně podrobná a jednoznačná, s čímž přichází určitá dávka složitosti zápisu. Diagramy se stávají více deskriptivní a více nahuštěné, a zároveň se vytváří diagramy pro každý aspekt softwarového produktu. Množství diagramů tak rapidně stoupá a jejich udržování se stává časově i zdrojově náročnější. Pokud navíc dodržujeme netriviální notaci (například symboly panáčků nebo obdélníků s lištami a popisky), je o něco časově náročnější dokreslit nový blok nebo ho najít v knihovně komponent.

C4 byl navržen právě proto, aby se předešlo složité symbolice a velkému počtu diagramů. Každý diagram je de facto blokový diagram, který se skládá z jednoduchých tvarů a šipek. UML produkuje velké množství diagramů pokrývající každý aspekt a část softwarového řešení (modelování do šířky), zatímco C4 produkuje maximálně čtyři diagramy, které jsou ale úplné (modelování do výšky).

Každý takový diagram odpovídá jedné úrovni abstrakce (Brown, 2024a). Ke každé z nich můžeme navíc přiřadit jeden princip návrhu, které jsem popsal v této práci dříve a které slouží k prevenci či mitigaci technického dluhu:

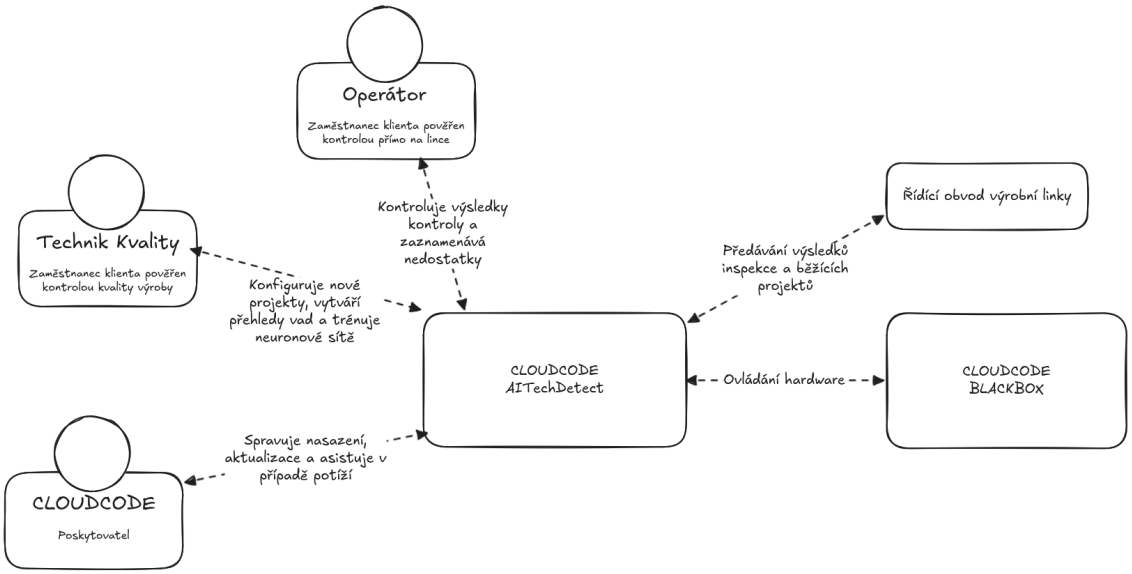
Úroveň C4	Název úrovně	Princip řízení kvality	Kapitola
Úroveň 4	Systémový kontext	Integrační vzory	2.5
Úroveň 3	Kontejnery	Architektonické vzory	2.4
Úroveň 2	Komponenty	Návrhové vzory	2.3
Úroveň 1	Kód	Čistý kód	2.2

Tabulka 4.1: Propojení úrovní modelu C4 s principy softwarového inženýrství

4.1 MODELOVÁNÍ SOFTWAROVÉ ARCHITEKTURY

4.1.1 Diagram systémového kontextu

Prvním krokem mé analýzy je zmapování systému v jeho přirozeném prostředí a definování hranic mezi stávajícím řešením a okolím. Tento diagram slouží k zachycení závislostí od řešení směrem k vnějším službám, ale i k zachycení závislostí od služeb směrem k řešení. Tím je možné vizualizovat, jak moc kritický je konkrétní systém v širším kontextu nasazených technologií (Brown, 2024b).



Obrázek 4.1: Diagram systémového kontextu

4.1.2 Diagram kontejnerů

4.1.3 Diagram komponent

4.2 MODELOVÁNÍ DOMÉNOVÝCH PROCESŮ

4.3 MODELOVÁNÍ PROCESU NASAZENÍ

4.4 VYHODNOCENÍ RIZIK

Po dokončení modelování softwarové architektury a doménových procesů vyvstala potřeba efektivně ohodnotit oblasti zatížené technickým dluhem. Mým cílem bylo nalézt metriku, která by minimalizovala subjektivní posouzení a umožnila rizika vyplývající z technického dluhu kvantifikovat. Právě soustředění se na riziko mě přivedlo k metodě FMEA (Failure Mode and Effects Analysis), které se věnoval ve své kvalifikační práci můj dlouholetý kolega z Univerzity Jana Evangelisty Purkyně v Ustí nad Labem, Bc. Matěj Vokál.

Metoda FMEA, etablovaná především v automobilovém průmyslu pro detekci vad při návrhu a produkci dílů, se nezaměřuje pouze na izolované nedostatky, ale na identifikaci kritických bodů selhání systému (Vokál, 2025). Tento analytický přístup jsem adaptoval pro potřeby zkoumaného softwarového řešení, neboť mi umožňuje hodnotit technický dluh prostřednictvím jasně definovaných kritérií.

Výstupem analýzy FMEA je matice, v níž každý řádek reprezentuje konkrétní systémovou položku a její potenciální způsob selhání (Failure Mode). Pro každý identifikovaný bod selhání je následně zpracována analýza vlivů, která postihuje jak bezprostřední lokální dopady v rámci dané komponenty, tak šíření poruchy do vyšších úrovní architektury a její celkový vliv na systém. Samotná kvantifikace rizik probíhá prostřednictvím definovaných hodnotících parametrů, mezi které patří pravděpodobnost výskytu, míra závažnosti, obtížnost detekce a průměrná doba do odhalení selhání. Z těchto faktorů je následně kalkulováno výsledné riziko, které slouží jako objektivní podklad pro prioritizaci a plánování následných nápravných opatření (Vokál, 2025).

Právě komplexní popis šíření selhání systémem a možnost objektivní kvantifikace rizik byly hlavními důvody, proč jsem si metodu FMEA vybral pro analýzu stávajícího řešení společnosti CLOUDCODE. Aby však bylo hodnocení konzistentní a nezávislé na subjektivním dojmu, je nezbytným prvním krokem definice hodnotících škál pro jednotlivé faktory rizika.

Prvním faktorem je pravděpodobnost výskytu selhání.

Stup.	Význam
1	Extrémně nepravděpodobné
2	Vyjímečné
3	Občasné
4	Pravděpodobné
5	Velmi časté

(a) Standardní FMEA škála pravdědpodobnosti výskytu selhání

Stup.	Definice
1	Extrémně nepravděpodobné
2	Vyjímečné
3	Občasné
4	Pravděpodobné
5	Velmi časté

(b) Adaptovaná škála pravděpodobnosti výskytu selhání

Tabulka 4.2: Škála pravděpodobnosti selhání

Stup.	Význam
1	**Bez dopadu**. Žádný vliv na spolehlivost a bezpečnost.
2	**Velmi nízká**. Produkt Zpozoruje pouze diskriminující zákazník.
3	**Nízká**. Produkt je lehce poškozen. Zpozoruje běžný zákazník.
4	**Kritické**. Produkt je vážně poškozen. Vede maximálně k jedné smrti.
5	**Katastrofální**. Produkt se stává nefunkčním a může vést k několika smrtím.

(a) Standardní FMEA škála vážnosti selhání

Stup.	Definice

(b) Adaptovaná škála vážnosti selhání

Tabulka 4.3: Škála vážnosti selhání

Stup.	Význam
1	**Jistá**.
2	**Skoro jistá**.
3	**Vysoká**.
4	**Průměrná**.
5	**Nízká**.
6	**Nedetekovatelná**

(a) Standardní FMEA škála odhalitelnosti selhání

Stup.	Definice

(b) Adaptovaná škála odhalitelnosti selhání

Tabulka 4.4: Škála odhalitelnosti selhání

5 ARCHITEKTURA NOVÉHO ŘEŠENÍ

6 ZÁVĚR

BIBLIOGRAFIE

BROWN, Simon, 2024a. *Introduction* [online]. 2024-08-14. [cit. 2026-03-13]. Dostupné z: https://c4model.com/introduction.

BROWN, Simon, 2024b. *System context diagram* [online]. 2024-08-14. [cit. 2026-03-13]. Dostupné z: https://c4model.com/diagrams/system-context.

DANĚK, Kevin, 2024. *Rapidní vývoj algoritmů strojového vidění* [online]. 2024-06-11. [cit. 2026-03-02]. Dostupné z: https://dspace.tul.cz/handle/15240/175790.

ELASTIC OBSERVABILITY TEAM, 2024. *The 3 pillars of observability: Unified logs, metrics, and traces* [online]. 2024-10-10. [cit. 2025-08-09]. Dostupné z: https://www.elastic.co/blog/3-pillars-of-observability.

FORRESTKNIGHT, 2025. *Everything You NEED to Know About TECH DEBT* [online]. 2025-07-24. [cit. 2025-08-10]. Dostupné z: https://www.youtube.com/watch?v=ukgmp6uxQJc.

FOWLER, Martin, 2009. *Technical Debt Quadrant* [online]. 2009-10-14. [cit. 2025-08-10]. Dostupné z: https://martinfowler.com/bliki/TechnicalDebtQuadrant.html.

MÄNTYLÄ, Mika, 2010. Empirical software evolvability - Code smells and human evaluations. In: s. 1–6. Dostupné z DOI: 10.1109/ICSM.2010.5609545.

MARTIN, Robert C.; COPLIEN, James O., 2009. *Clean code: a handbook of agile software craftsmanship*. Prentice Hall. ISBN 9780132350884.

MICHIGAN TECHNOLOGICAL UNIVERSITY, 2025. *What is software engineering?* [online]. 2025-04-09. [cit. 2026-03-03]. Dostupné z: https://www.mtu.edu/cs/undergraduate/software/what/.

PEREZ, Javier, 2011. *Refactoring Planning for Design Smell Correction in Object-Oriented Software*. Dis. pr.

SHVETS, Alexander, 2018a. *Code Smells* [online]. 2018-02-08. [cit. 2026-03-03]. Dostupné z: https://refactoring.guru/refactoring/smells.

SHVETS, Alexander, 2018b. *Code Smells* [online]. 2018-02-08. [cit. 2026-03-03]. Dostupné z: https://refactoring.guru/design-patterns/what-is-pattern.

SHVETS, Alexandr, 2018. *Classification of patterns* [online]. 2018-02-08. [cit. 2026-03-20]. Dostupné z: https://refactoring.guru/design-patterns/classification.

ŠPÁNEK, Roman, 2022. *Přednáška č. 5: UML I.* [Přednáška v rámci předmětu Softwarové inženýrství]. [cit. 2026-03-13]. Technická univerzita v Liberci.

TEAM ASANA, 2025. *What is technical debt? How to pay it off (with examples)* [online]. 2025-06-25. [cit. 2025-08-10]. Dostupné z: https://asana.com/resources/technical-debt.

TORVALDS, Linus, 2007. *Re: Coding Style* [online]. 2007-06-15. [cit. 2026-03-03]. Dostupné z: https://yarchive.net/comp/linux/coding_style.html.

VOKÁL, Matěj, 2025. *Zpracování harmonizované FMEA pro kryt a podlahu zavazadlového prostoru vozu Škoda Elroq dodávaného IMG AUTOMOTIVE s.r.o.* [online]. [cit. 2026-03-14]. Bakalářská práce. Univerzita Jana Evangelisty Purkyně. Fakulta strojního inženýrství. Ústí nad Labem.